

Stock Analysis Pipeline

Spécification Technique v1.1

Version:	1.1
Date:	2026-05-18
Status:	Brouillon — Review externe
Auteurs:	Hermes + Codex CLI
Nature:	Référence technique vérifiée (29 endpoints audités)

Document de référence technique pour Stock Analysis Pipeline. Décrit la stack (Python/FastAPI + React/Vite + ReportLab + Cloudflare), l'architecture des modules, les 29 routes API auditées, le modèle de scoring 40 points, la gestion des erreurs, les sources de données, la sécurité, les tests, le déploiement, et les pièges techniques connus.

1. Stack technique

Couche	Technologie	Version
Backend	Python + FastAPI	3.11+ / 0.115+
Frontend	React + Vite	React 18+
PDF Engine	ReportLab	4.x
Scoring	Rules engine	40 pts, 6 catégories
Sources	yfinance, Finnhub, EDGAR, SA, Tavily, AlphaVantage	—
Tunnel	Cloudflare Tunnel (named)	e92cfcfb
Runtime	WSL2 + venv	—
Tests	pytest + Playwright	—

2. Architecture

Navigateur → sa.cedlabusa.net → Cloudflare Tunnel → Backend FastAPI (localhost:8780) → StaticFiles (dist/) + Router API (29 routes, 6 groupes) → Sources externes (yfinance, Finnhub, EDGAR, Seeking Alpha, Tavily, Alpha Vantage).

2.1 Modules backend

```
backend/  
  main.py          # Routes (29 endpoints) + StaticFiles  
  pipeline.py      # Orchestration 9 étapes  
  models.py        # Pydantic models  
  sources_collector.py # Multi-source (yfinance, Finnhub, EDGAR)  
  scorer.py        # Scoring rules-based 40 pts  
  transcript_finder.py # Transcripts (SA, web)  
  edgar_extractor.py # SEC EDGAR parsing  
  feedback_store.py # Gestion feedback  
  earnings_deep_dive/  
    schemas.py      # DeepDiveRequest/Response  
    collector.py     # Collecte deep-dive  
    renderer.py      # ReportLab PDF renderer  
  cache/           # Volatil  
  logs/            # JSON structurés  
  analyses/        # Archivage
```

3. Routes API (29 endpoints, audités 2026-05-18)

Vérifié via `grep -n '@app\.' backend/main.py`

3.1 Health & Debug (3 routes)

ID	Méthode	Path
API-001	GET	/api/health

API-002	GET	/api/debug/yf-cache/{ticker}
API-003	GET	/api/debug/sources

3.2 Analyse (4 routes)

ID	Méthode	Path
API-010	POST	/api/analyze
API-011	POST	/api/analyze/async
API-012	GET	/api/analyze/job/{job_id}
API-013	GET	/api/analyze/{ticker}/download

3.3 Earnings Deep-dive (6 routes)

ID	Méthode	Path
API-020	GET	/api/earnings/quarters/{ticker}
API-021	POST	/api/earnings/deep-dive
API-022	HEAD	/api/report/{ticker}/pdf
API-023	GET	/api/report/{ticker}/pdf
API-024	HEAD	/api/report/{ticker}
API-025	GET	/api/report/{ticker}

4. Modèle de scoring (40 points)

Catégorie	ID	Points	Données
Financial Health	SC-FIN	/10	Dette/equity, current ratio, FCF
Growth	SC-GRW	/10	Revenue growth YoY, EPS trend
Valuation	SC-VAL	/8	PER, PEG, P/B, EV/EBITDA
Management	SC-MGT	/5	Tone analysis, insider trades
Moat	SC-MOA	/4	Gross margin, market share
Sentiment	SC-SEN	/3	News sentiment, analyst ratings

BUY ≥ 28 | HOLD 18-27 | SELL < 18

5. Gestion des erreurs

Erreur	HTTP	Comportement
Ticker invalide	400	{"error": "Invalid ticker"}
Ticker non trouvé	404	{"error": "Ticker not found"}
Analyse en cours	409	{"error": "Analysis in progress", "job_id": "..."}
Batch > 10 tickers	400	{"error": "Max 10 tickers per batch"}
API source timeout	503	Continue avec sources restantes
Rate limit source	—	Retry 3x backoff, skip si échec
PDF non trouvé	404	{"error": "No report for this ticker"}

Donnée manquante

—

"DONNÉE NON DISPONIBLE" dans PDF

6. Sources de données

Source	Rate limit	Timeout	Fallback
Yahoo Finance	~5 req/s	15 s	Finnhub
Finnhub	60 req/min	10 s	EDGAR
SEC EDGAR	~10 req/s	20 s	Aucun
Seeking Alpha	Variable	25 s	Tavily
Tavily	1000/mois	10 s	DDG (dégradé)
Alpha Vantage	25/jour	10 s	Cache+Finnhub

7. Sécurité

- SEC-001 : Secrets dans .env, jamais commités (.gitignore)
- SEC-002 : X-API-Key (CED_CONTROL_KEY) requis pour write endpoints
- SEC-003 : CORS restreint (sa.cedlabusa.net, localhost)
- SEC-004 : Logs sans secrets (JSON structuré)
- SEC-005 : Cloudflare TLS (named tunnel, chiffrement bout en bout)

8. Tests

Fichier	Type	Couvre
tests/test_pipeline.py	Unitaire	Étapes pipeline
tests/test_scorer.py	Unitaire	Scoring 40 pts
tests/test_sources_collector.py	Unitaire	Collecte multi-source
tests/test_edgar.py	Unitaire	SEC filings
tests/test_renderer.py	Unitaire	Génération PDF
tests_e2e/test_sa_recette.py	E2E Playwright	Parcours complet
tests_e2e/conftest.py	Fixtures	Setup navigateur

9. Déploiement

9.1 Backend

```
cd backend && source .venv/bin/activate
PYTHONPATH=.. uvicorn main:app --host 0.0.0.0 --port 8780
```

9.2 Frontend

```
cd frontend && npm run build
```

9.3 Tunnel

```
cloudflared tunnel run e92cfcfb-86b6-4e36-9669-2120cad8bed3
```

10. Pièges techniques connus

ID	Symptôme	Fix
PIT-001	Score erroné après EDGAR	Sync financials+fin (commit 3cf8f9b)
PIT-002	Quick tunnel parasite	ps aux grep cloudflared → kill quick
PIT-003	JS 404 /stock-analysis/stock-analysis/	Accès root URL
PIT-004	404 JS/CSS	npm run build avant start backend
PIT-005	PDF None	Helpers BEFORE render function
PIT-006	Port Windows ghost	powershell Get-NetTCPConnection
PIT-007	Frontend stale	rm -rf dist/ cache/ && npm run build